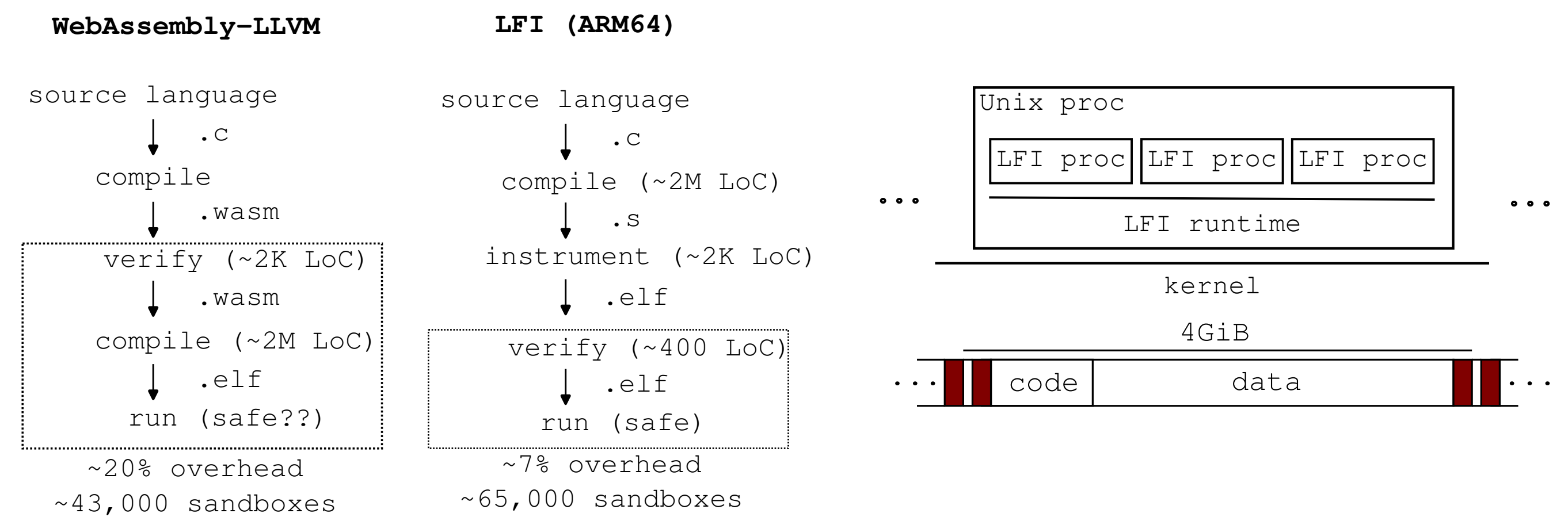


Extending LFI Machine Code Validation with Decision Diagrams

Yavor Litchev Mentors: Zachary Yedidia, David Mazières

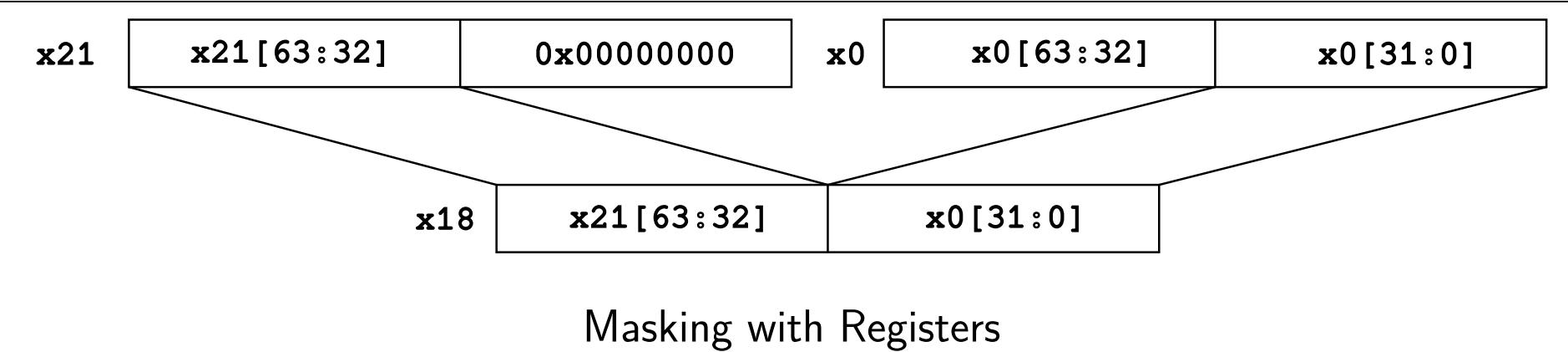
Introduction and Motivation

Process isolation is critical for the security of computer systems. Hardware protections are typically used, but can be slow when changing protection domains. Software-based Fault Isolation (SFI) solves this issue, and Lightweight Fault Isolation (LFI), the basis of this research project, is a versatile and performance orientated example of this.



Extension 1: RISC-V

RISC-V is a popular instruction set architecture (ISA) that has many positive qualities, such as being open source and power efficient.



- RISC-V LFI system functions by compiling source code while reserving registers x18, x21, and x22. load, store, jump, and other unsafe instructions are detected via pattern matching, and guard instructions are placed (see examples below)
- If instructions are not 4 byte aligned (compressed instructions), bundles can be used to safeguard jumps

Loads:

```
ld t0, (t1)
>>>
add.uw x18, t1, x21
ld t0, (x18)
```

Arithmetic:

```
addi sp, sp, 16
>>>
addi x22, sp, 16
add.uw sp, x22, x21
```

Stores:

```
sd t0, 16(t1)
>>>
add.uw x18, t1, x21
sd t0, 16(x18)
```

Special Registers:

```
ld sp, (a0)
>>>
add.uw x18, a0, x21
ld x22, (x18)
add.uw sp, x22, x21
```

Jumps/Branches:

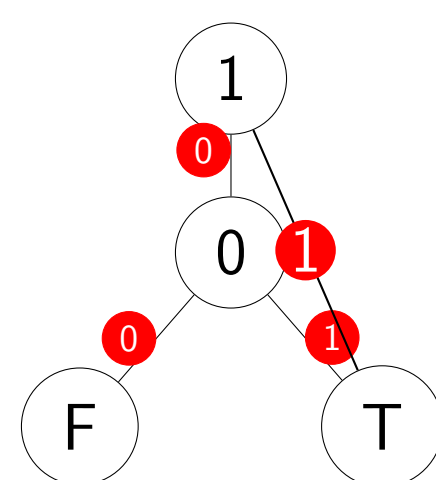
```
jr x0
>>>
add.uw x18, x0, x21
jr x18
```

System Calls:

```
ecall
>>>
sd ra, (sp)
ld ra, (x21)
jalr ra
ld x22, (sp)
add.uw ra, x22, x21
```

Motivation for Verifier

- It may be expensive for server to perform compilation, so one may offload safe compilation process to the client, and have the server verify code safety
- Since registers are reserved to house safe addresses, the safety of a binary is context free
- Disassemblers trivially solve this, but are slow, so we leverage binary decision trees (BDD) to perform a speedup

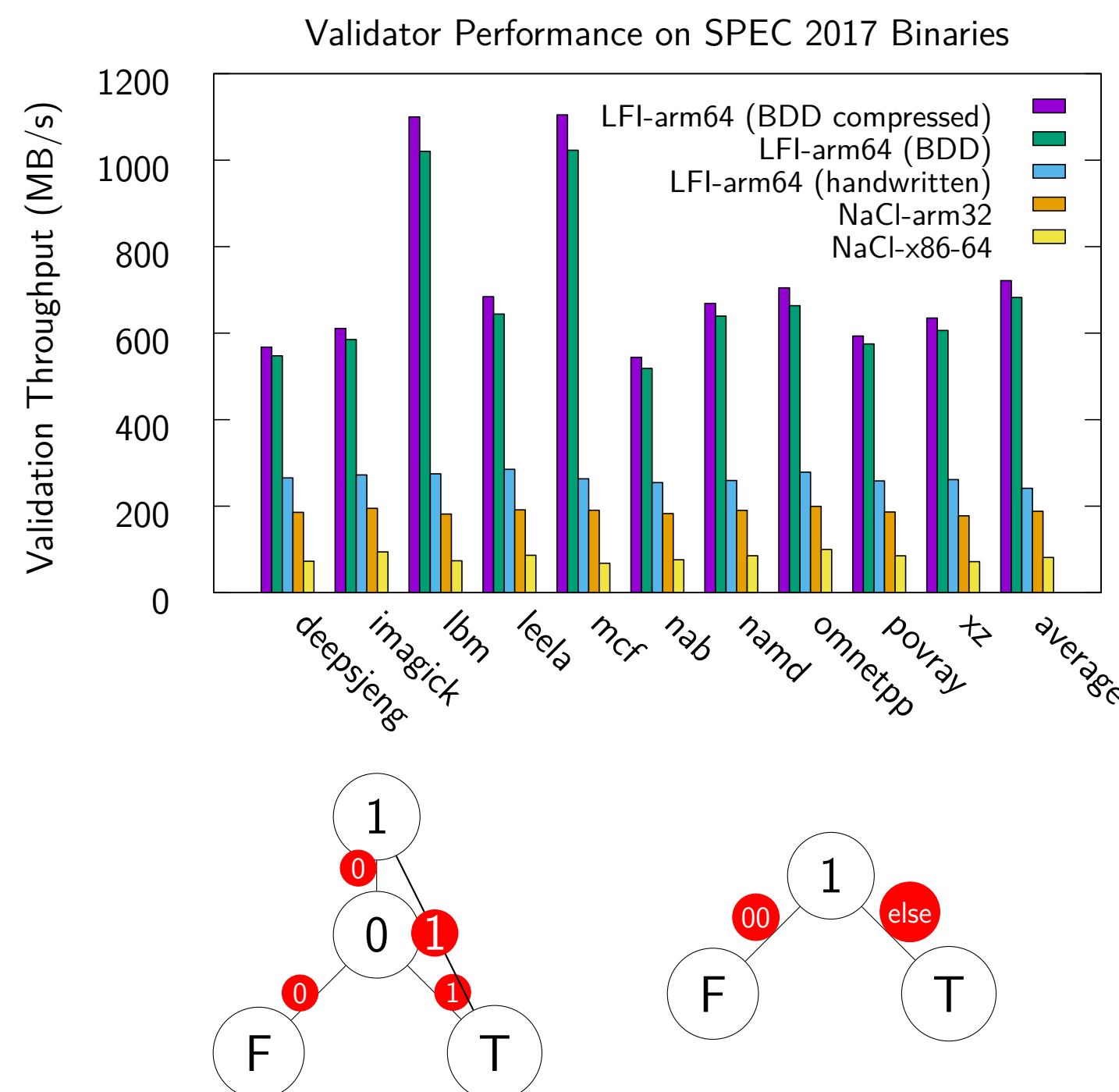


Extension 2: Binary Decision Diagram Compression

- Although BDDs are much faster than disassemblers, single bit checking is inefficient
- Checking multiple consecutive bits at a time allows for the verifier to reach a terminal node faster
- Insight: identify redundant checks present in tree that lead to the same two nodes anyways

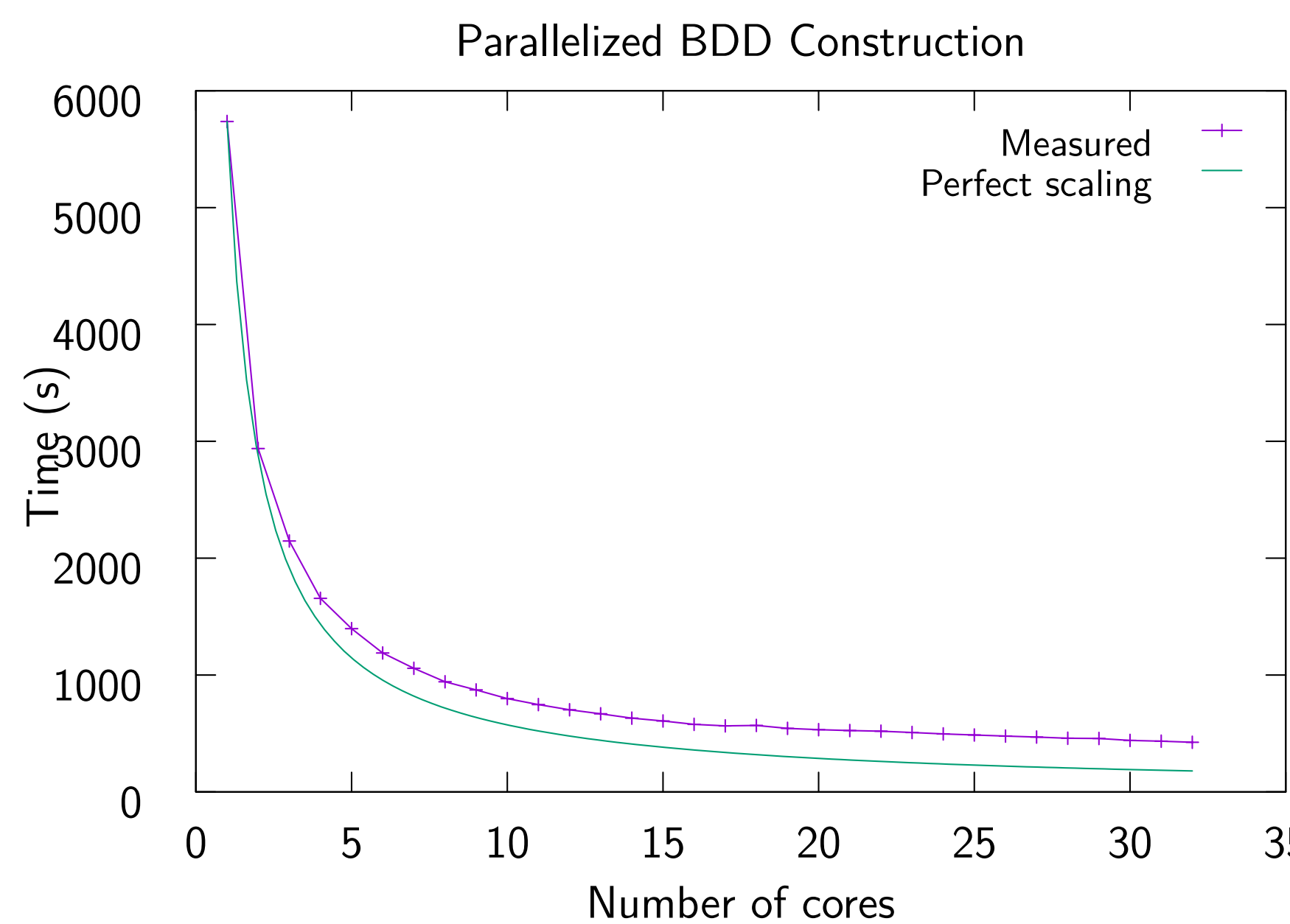
Algorithm:

- FOR each Node in Nodes
1. Locate both child Nodes
2. Check if one of the children of one of the child Nodes (grand child of the original Node) also matches the other child Node of the original Node. If no match, skip case
3. If matched, such a node is a fixed node
4. WHILE the child Nodes have at least one Node that is equal to the fixed Node, continue down the tree, keeping track of which bits are needed to reach the child Node that is not the fixed Node
5. Store the string of bits needed to reach the non fixed Node, and if the bit string is rejected, go to the fixed Node instead



Extension 3: Parallel BDD Construction

- The BDD library that was used, BuDDy, functions by creating a BDD object, creating a set number of variables (32 for ARM instructions), and denoting if a bit pattern for a particular instruction should be AND or OR'd with the tree (i.e. with OR, the bit pattern should return true along with the other bit patterns encompassed in the BDD)
- This process can be quite slow. A key observation that can be made regarding speeding up the process is that separate BDD trees can be constructed for different instruction sets, and each of the sub BDD's can be OR'ed together



- Valid instructions are not necessarily evenly distributed throughout the instruction range
- Therefore, we created a multithreaded environment to preemptively detect valid instructions
- The first pass crafts a tally of the total number of valid instructions
- The second pass identifies the correct instruction range for each subprocess, using the total as a guide

Relative Sizes of Different Verifier Systems

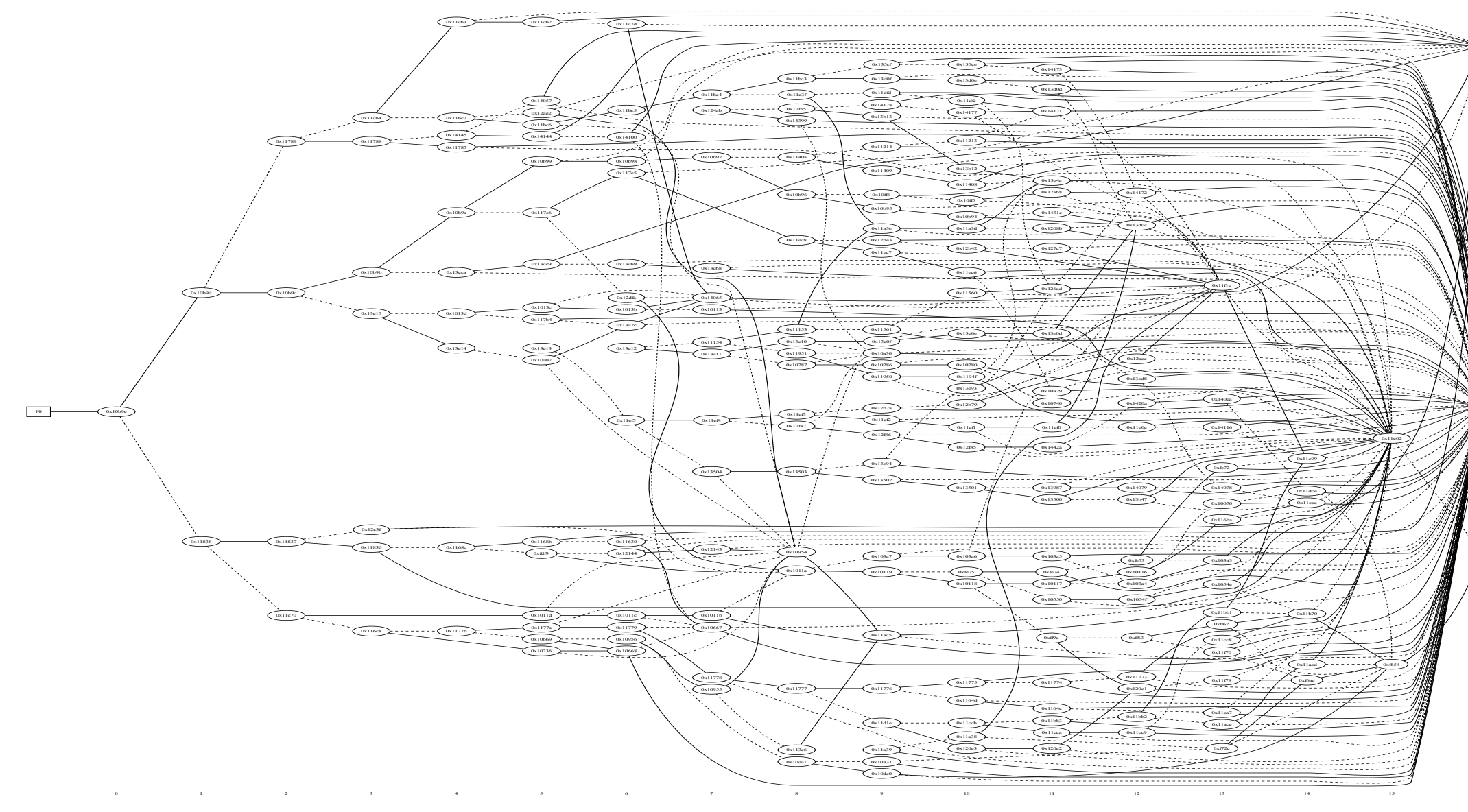
LFI-arm64 (BDD): 26KB

LFI-arm64 (disassembler based): 80KB

NaCl-arm32: 89KB

Extension 4: Algebraic Decision Diagram for x86-64

Algebraic Decision Diagrams (ADD) allow us to have multiple terminal nodes, so we can encode instruction length along with the validity of an instruction.



Example ADD for x86 instructions that are 2 bytes or shorter

We have also generated an ADD for the full x86 instruction set, containing about 13,000 nodes. We generated instructions by exhaustively enumerating non-immediate inputs to an encoder, resulting in around 700M instructions.

Extension 5: Machine Readable Spec, Pseudocode Extraction

- We wish to read in ARM's official machine readable specification (XML) format to create custom disassemblers
- One program generates C code decoding any ARM instruction
- Another extracts ASL pseudocode and creates decoding and operation functions (see right)

```
// Name: add_addsub_imm
function Decode(Rd, Rn, imm12, sh, S, op, sf)
constant integer d = UInt(Rd);
constant integer n = UInt(Rn);
constant integer datasize = 32 << UInt(sf);
constant bits(24) imm = if sh == '0'
    then Zeros(12):imm12 else imm12:Zeros(12);
```

Conclusion and Future Research

This project seeked to extend the already highly versatile LFI system, and in this respect we were able to make the LFI system more efficient in verification, more flexible in which ISA's can be used, and more able to accomodate changes in ISA architectures. Future directions for research include:

- Creating a compiler that translates ASL pseudocode to C code, so as to perform mock execution of ARM instructions based on the machine readable spec
- Translate unsafe binaries to safe binaries, via disassembling binaries and adding guard instructions
- Experimenting with other forms of verification besides BDDs

References

- [1] Buddy: A bdd package.
https://buddy.sourceforge.net/manual/main.html, 2007.
- [2] Zachary Yedidia.
- Lightweight fault isolation: Practical, efficient, and secure software sandboxing.
In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 649–665, 2024.
- [3] Bennet Yee, David Sehr, Gregory Dardyk, J Bradley Chen, Robert Muth, Tavis Ormandy, Shiki Okasaka, Neha Narula, and Nicholas Fullagar.
Native client: A sandbox for portable, untrusted x86 native code.
Communications of the ACM, 53(1):91–99, 2010.